

Nykaa Fit

A four-question size recommender on the product page, built to answer the one thing standing between a shopper and the buy button: *which size do I take, and how will it sit on me?*

SCOPE

Women's dresses · 10 products

ENGINE

Deterministic · no model, no API

TESTS

40 passing

STACK

React 19 · TypeScript · Vite

The question the page has to answer

A shopper looking at a dress online can read the fabric, the length and the return policy. What she cannot read is whether the M will fit her.

So she does one of three things: guesses, opens a size chart she can't map onto her own body, or orders two sizes and sends one back. The first two lose sales at the size selector. The third turns a sale into a return, and a size-related return is the most expensive kind — the item comes back, the shipping is paid twice, and the shopper trusts the next product page a little less.

Nykaa Fit answers the question at the moment it gets asked: on the product page, directly beneath the size buttons.

The journey

- 01 Find My Fit**
Sits under the size selector as an invitation, never a gate. The size chips above it work exactly as before for anyone who ignores it.
 - 02 Four questions**
Height, weight, gender, preferred fit. Age and body shape are optional and collapsed. Under a minute. No photographs, no scanning, no chat.
-

-
- 03 **A size, with a confidence**
"L is your recommended size · 81% confidence · Regular fit", plus what that means at the bust, waist and hip.
-
- 04 **Why we recommend L**
Every factor that moved the answer, and which direction it pushed. The shopper can disagree with a reason; she can only distrust a verdict.
-
- 05 **Select L**
One tap applies it to the real size selector, which marks it "✓ L is your recommended size". Any other size stays selectable.
-
- 06 **Every dress after this one**
The profile is saved on the device. The next product page shows a recommendation straight away — and, crucially, often a different size.
-

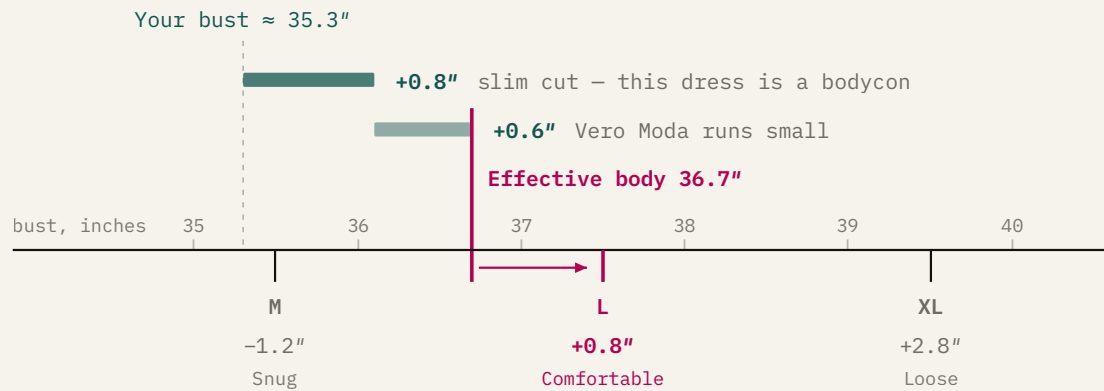
How the size is chosen

No model, no API call, no randomness. The same profile and the same product always produce the same answer — which is exactly what makes it testable in CI and explainable to a shopper.

```
estimated body    height, weight, gender, optional age / shape
+ product cut     slim +0.8"   regular 0    relaxed -0.8"   oversized -1.6"
+ preferred fit    slim -0.7"   regular 0    relaxed +0.9"
+ brand sizing     runs small · true to size · runs large
```

= effective body → nearest size in this brand's own chart

The trick is that the adjustments move the *body*, not the chart. A slim cut on a given body needs a larger size to feel the same as a regular one, so the body we look up gets bigger. One comparison then drives both the ranking and the words the shopper reads.



One shopper, 165 cm and 60 kg, on the Vero Moda Ruched Bodycon. Her estimated bust sits nearest to M on the raw numbers — but the slim cut and the brand's run-small behaviour shift the body being looked up by 1.4", and L becomes the closest size. This is the whole mechanism: the adjustments move the body, and the chart stays where it is.

Estimating the body

Height and weight become bust, waist and hip through a girth index, $\sqrt[3]{(\text{kg} / \text{m})}$.

Approximate the torso as a cylinder of roughly constant density and its volume goes as $\text{height} \times \text{girth}^2$, so circumference scales with the square root of mass over height. Every coefficient is anchored to a stated reference body rather than tuned by hand, so the behaviour can be read off the source.

Sizes are then ranked by weighted distance — bust **0.45**, waist **0.35**, hip **0.20**, because bust drives dress fit hardest.

Why the same shopper gets four different sizes

This is the part that decides whether the feature is worth building. A recommender that maps weight onto S/M/L is worthless — a shopper already knows she is "usually a medium". The value is in the disagreement *between* brands.

So every brand carries two separate things: its own published chart (a block plus a grade per size step) *and* how the finished garment actually behaves against that chart. A brand can publish an accurate chart and still sew tight.

ONE SAVED PROFILE — 165 CM, 60 KG, PREFERS A REGULAR FIT

SIZE	DRESS	CUT	BRAND SIZING
XS	W for Woman — Colourblock Tiered Midi	relaxed	runs large
S	Libas — Floral Print Anarkali Maxi	regular	runs large
S	Global Desi — Tiered Cotton Boho Maxi	relaxed	runs large
S	Biba — Chikankari Cotton A-Line	regular	true to size
M	AND — Belted Shirt-Style Midi	regular	true to size
L	Vero Moda — Ruched Bodycon Midi	slim	runs small
L	Kazo — Sequin Embellished Party	slim	runs small

Four sizes across seven dresses, from one profile, with no questions re-asked. That spread is the product. Without it there is nothing here a shopper couldn't already guess.

Earning the trust rather than asking for it

A size recommendation is a claim about someone's body made from two numbers. It has to be offered honestly or it does more harm than the guessing it replaces.

Confidence is bounded on purpose

Confidence blends how well the winning size fits with how clearly it beats the runner-up. It is floored at **50%** and **capped at 94%**. The copy says "should" and "estimate", never "guaranteed" or "perfect fit" — there is a test that fails the build if those words appear.

Nothing is hidden and nothing is locked

"Why we recommend L" lists every input and which direction it pushed, in inches. "Compare sizes" shows what sizing up or down would actually change, so the hesitant shopper resolves the doubt on the page instead of leaving to think about it. And the recommendation is a marked chip, not a locked one — any size stays one tap away.

When the answer is out of stock

The engine ranks every size, then falls back to the nearest one that can actually be bought and says so plainly: *"Your recommended size XXL is currently unavailable. XL is the closest available option."* Silently recommending second-best would be the easy thing and the wrong one.

Privacy. Height and weight live in `localStorage` and nowhere else. The feature makes no network call at all. Analytics carry the *decision* — which size was recommended, which was chosen — never a measurement, and a test asserts that no event payload can contain one.

How we would know it worked

The hypothesis is that fit guidance lifts conversion by removing sizing uncertainty, and cuts size-related returns by improving the choice. Neither is assumed anywhere in the build. The app's job is to emit the events an analysis would consume.

product_view Top of the funnel, fired in both arms	fit_cta_clicked Did the entry point get noticed	fit_profile_started Did she begin the form
fit_profile_completed Is four questions short enough	fit_recommendation_shown Impressions, deduplicated per view	fit_recommendation_accepted Did she take the size we picked
size_selected Carries recommended vs manual source	add_to_bag The primary conversion event	purchase One per line, joinable to returns later

The experiment

CONTROL The existing product page, untouched. No fit surface renders at all — but every event still fires , stamped with the bucket.	TREATMENT The same page plus Nykaa Fit. Primary metric: product-view → add-to-bag conversion , compared between the two arms.
---	--

Assignment is sticky per browser and reversible from the URL (`?fit=off`) or from an internal Fit Lab page, which shows the bucket, the funnel and the metric definitions computed from that browser's own events. All nine events go through one abstraction with pluggable sinks, so connecting a real provider later is a single `registerSink()` call and touches no call site.

No results are claimed anywhere. With no traffic the metrics read “no data yet” rather than zero. The four guardrails — bounce, checkout abandonment, page performance, size-related returns — are named alongside what each would need, because none of them is instrumented in this prototype and pretending otherwise would be the easiest lie to tell.

How it is built

The whole feature is one folder. The engine is plain TypeScript with no React and no network, which is why forty tests can cover it directly — the ten required scenarios plus body modelling, persistence, analytics and event de-duplication.

```
src/features/nykaa-fit/  
  components/  FitBlock (orchestrator), FitCTA, FitProfileForm,  
               FitResult, FitProfileSummary, SizeComparison, FitPanel  
  engine/      fitEngine, scoring, bodyModel ← pure, no React  
  analytics/    fitAnalytics (pluggable sinks)  
  experiment/   variant assignment  
  utils/        fitStorage, eligibility, hooks  
  types/        fitTypes
```

The store knows about the feature in exactly two places: one block on the product page and one optional prop on the size selector. Eligibility is a single predicate — widening the experiment past dresses means extending it and adding brand charts for the new category, with no UI change.

Two bugs worth mentioning

Double-counted impressions. React StrictMode deliberately invokes effects twice in development, and the component ref guarding the view event was recreated by that remount — so `product_view` fired twice and corrupted the funnel. The guard has to live at module scope, keyed by the router’s location key. Two regression tests hold it.

The form overflowed on mobile. A text input’s intrinsic width is about twenty characters, which was enough to blow out the grid tracks inside a 375 px sheet. Fixed by letting the grid items shrink.

What I would push back on in a review

The size charts are plausible mock data, not the brands' real published charts. The engine is only ever as accurate as those inputs. A real build starts by ingesting actual brand charts and, more valuably, historical return reasons.

The body model is a heuristic, not an anthropometric study. It is presented that way throughout, but two numbers will never separate two people of the same height and weight with very different frames. Body shape is the optional input that partially addresses this.

Adoption under-counts returning shoppers, who see the inline recommendation card without ever clicking the entry point. The metric is labelled precisely for that reason.

UI paths were verified by driving a real browser, not by automated DOM tests. Adding jsdom and Testing Library is the obvious next commit.

Confidence is a calibrated-looking number that has never been calibrated. It is derived from fit distance and separation, not from observed outcomes. Once returns data exists, that is the first thing to replace.

Running it

```
npm install
npm run dev      # http://localhost:5173
npm test         # 40 tests
```

Open any dress from `/c/dresses` for the fit flow, or `/fit-lab` for the experiment wiring and the raw event stream. Append `?fit=off` to any product page to see the control arm.
